
Design Specification

Programmable Interrupt Controller with Advanced Scheduling

Sixteen hardware sources and sixteen software channels,
priority banding, hardware nesting and deadline escalation,
configured over AXI4-Lite (Advanced eXtensible Interface,
revision 4, Lite subset)

Document	Design Specification — Programmable Interrupt Controller
Document type	Design specification
Subject	Programmable interrupt controller, sixteen sources, hardware nesting
Revision	1.2
Date	16 September 2026
Author	Bunea Cosmin-Andrei
Status	Released for review

Contents

1	Introduction	3
1.1	Purpose of the document	3
1.2	Validity of the document	3
1.3	Definitions and terms of abbreviations	3
1.4	Requirements and implementation decisions	4
1.5	Relationship with other documents	4
1.6	Notes on the notation used	4
2	Specifications and system integration	5
3	Overview of overall structure	6
3.1	Operating principle	6
3.2	Block structure	7
3.3	Source life cycle	7
3.4	Priority	7
3.5	Data and control flow	8
4	Interfaces	8
4.1	Hardware interface	8
4.1.1	Clock and reset	9
4.1.2	Interrupt source interface	9
4.1.3	Processor interface	10
4.1.4	Register access interface	11
4.2	Software interface	11
4.2.1	Register map	11
4.2.2	Register fields	12
4.2.3	Programming sequences	15
5	Reuse and reusability	16
6	Design of submodules	17
6.1	Source conditioning	17
6.1.1	Interface description	17
6.1.2	Functional description	17
6.2	Priority key formation	17
6.2.1	Interface description	17
6.2.2	Functional description	17
6.3	Deadline monitoring and escalation	18
6.3.1	Interface description	18
6.3.2	Functional description	18
6.4	Priority resolver	19
6.4.1	Interface description	19
6.4.2	Functional description	19
6.5	Nesting stack and depth limit	19
6.5.1	Interface description	19
6.5.2	Functional description	19
6.6	Spurious claim detection	20
6.6.1	Interface description	20
6.6.2	Functional description	20

6.7	Event and status flags	20
6.7.1	Interface description	20
6.7.2	Functional description	20
6.8	Register interface	21
6.8.1	Interface description	21
6.8.2	Functional description	21
6.9	RTL organisation	21
7	List of bugs and workarounds	21
8	Annex	22
8.1	Reset state	22
8.2	Timing summary	23

1 Introduction

1.1 Purpose of the document

This document specifies the design of the programmable interrupt controller: its architecture, the responsibilities of its functional blocks, the interfaces it presents to the rest of the system, its software-visible register set, and the behaviour a system integrator and a driver author must rely on.

1.2 Validity of the document

The document is valid for the interrupt controller as currently implemented: sixteen hardware source lines, sixteen software channels sharing the same sixteen priority slots, one prioritised request presented to one processor, and one AXI4-Lite slave port for configuration and status.

Blocks the controller connects to are described only as far as the contract at the boundary between them and the controller requires.

1.3 Definitions and terms of abbreviations

Term	Meaning
AXI4-Lite	Advanced eXtensible Interface, revision 4, Lite subset. The bus protocol of the register port.
Band	One of four priority groups. A source belongs to one band; the band contributes the most significant part of its priority.
Claim	The single-cycle pulse with which the processor announces that it has entered the handler for the source currently being offered.
Deadline	The number of cycles a source may remain pending before its priority is escalated.
End of interrupt	The single-cycle pulse with which the processor announces that one interrupt handler is returning.
Escalation	The automatic move of a pending source to a more urgent band after its deadline has elapsed.
In service	The state of a source between its claim and the end of interrupt that releases it. An in-service source occupies one entry of the nesting stack.
Intra-band priority	The four-bit priority that orders sources within one band.
Nesting depth	The number of interrupts currently in service, that is, the number of occupied entries of the nesting stack.
Offer	The request the controller presents to the processor, consisting of the request line and the source number of the winning source.
Pending	The state of a source that has an enabled request and is not in service.
Preemption	The claim of a source more urgent than the one already in service, which opens a further nesting level rather than replacing the current one.
RO	Read only. Writes are rejected.
RW	Read/write.
RW1C	Read; write one to clear. Writing a one to a bit clears it, writing a zero leaves it alone.
Slot	One of the sixteen priority positions. A hardware line and a software channel of the same number share one slot.
Source	One slot considered as an interrupt requester, regardless of whether the request came from the hardware line or the software channel.

Term	Meaning
Spurious claim	A claim that lands after the request that caused the offer has already gone away.
Urgency	The two-bit value a band carries. A band with a higher urgency outranks a band with a lower one, whatever their numbers.
WARL	Write any value, read legal value. A write of an out-of-range value is accepted on the bus and stored as a legal value.

1.4 Requirements and implementation decisions

The original Siemens briefs are kept in `cpu/docs/brief/`: *RV32I 3-Stage Pipeline CPU* and *Programmable Interrupt Controller with Advanced Scheduling*. They define the project scope. Instruction and CSR semantics are checked against the cited RISC-V manuals, and bus handshakes against the AXI specification. These authored specifications describe the implementation; they are not independent evidence that the implementation is correct.

Details absent from the briefs, including the register map, bus error policy and the sixteen-source claim/EOI interface, are project decisions. The CPU brief's eight-source interface and the PIC brief's sixteen-source interface still require a common interface agreement; the implemented contract is stated explicitly in the design specifications.

1.5 Relationship with other documents

Document	Relationship
Verification Specification — Programmable Interrupt Controller	Specifies the verification environment applied to the design described here, and what that environment establishes.
Design Specification — RV32I Processor Core	Specifies the processor at the other end of the interrupt interface: how it decides to accept an offer, when it emits the claim and the end-of-interrupt pulses, and which of its registers carry the enable mask and the pending set exchanged with the controller.
AMBA AXI and ACE Protocol Specification, ARM IHI 0022E	Defines the AXI4-Lite protocol used by the register port.

1.6 Notes on the notation used

- Fixed-width type denotes a signal, a register, a field or a literal value, for example **AWVALID** or **INT_ENABLE**.
- Italic type marks emphasis, or a term where a sentence defines it, as in a *level* source. Bold type marks a run-in heading.
- Numeric constants are hexadecimal when prefixed **0x**, otherwise decimal. Bit ranges are written most significant bit first, as **[31:16]**.
- Register offsets are byte offsets from the base address at which the block is decoded.
- A per-source register name written with an **x**, such as **SRCx_CONFIG**, stands for all sixteen instances of that register.

- “The controller” means the block specified by this document. “The processor” means the core that consumes its offers. “The system” means whatever the controller is integrated into.
- In interface tables, direction is given from the point of view of the controller: an output leaves the controller.

2 Specifications and system integration

The controller resolves interrupt requests from sixteen sources and presents one of them at a time to a single processor. It holds no data path, initiates no bus transaction, and cannot operate standalone. A system integrating it must supply the following.

Table 2: What the controller requires from the system

Resource	Requirement
Clock and reset	One clock. One asynchronous, active-low reset. The controller contains no clock-domain crossing and no internally generated clock or reset.
Address window	A 256-byte window on an AXI4-Lite interconnect. The block decodes address bits [7:2] and nothing above them, so a window wider than 256 bytes produces aliases of the whole register file rather than an error response.
Register master	An AXI4-Lite master with at most one outstanding transaction per direction. The slave accepts a second address only after the response to the first has been taken.
Source lines	Up to sixteen level or pulse signals in the same clock domain, synchronous to the controller’s clock. The controller attaches no meaning to a particular line number; which device drives which line is a system decision that only the software configuration has to agree with. Unused lines must be tied low.
Processor	A core that samples the offer, replies with a single-cycle claim, and emits a single-cycle end of interrupt when a handler returns. The exact timing relationship is specified in Section 4.1.3.

Sources are not the same thing as lines. Each of the sixteen slots can be raised by its hardware line or by a software write, which gives thirty-two logical request origins over sixteen priority positions. Once a request is in the slot the controller does not distinguish the two: they carry the same priority, the same deadline and the same status bits.

The controller is written in Verilog–2005 within the synthesisable subset, but the current target is behavioural simulation; see Section 7.

Constraints the integrator must respect. Three properties of the interface are contracts rather than preferences, and none of them is enforced by hardware.

The claim and the end-of-interrupt pulse must never be asserted in the same cycle. The depth counter treats the claim as the winning event, so a coincidence would increment the depth for an interrupt that was simultaneously supposed to be released, and the accounting would never recover.

The claim must arrive exactly one cycle after the cycle in which the processor sampled the offer. The controller claims the source number as it stood one cycle earlier, precisely so that a more urgent source arriving in the meantime cannot change which source the claim refers to. A

master that delays its claim by an arbitrary number of cycles after sampling will claim the wrong source.

A claim that arrives with no offer outstanding is accepted. It pushes the source number left over from the last valid offer, and flags that level as spurious if the source in question is no longer requesting. The accounting stays consistent either way, so an unprompted claim costs one wasted handler entry rather than leaving the controller unable to track its levels. The processor is still expected not to claim unprompted.

3 Overview of overall structure

3.1 Operating principle

A request travels through six stages between the source and the handler.

Conditioning. Each slot turns its hardware line into a request according to its trigger type: a level source requests while its line is high, an edge source requests from the rising edge of its line until the request is consumed. The software channel of the same slot is merged in at this point.

Enabling. The merged request is qualified by the per-source enable in `INT_ENABLE`. A source that is not enabled produces nothing: it is not offered, it does not appear in the pending set, and its deadline does not run.

Key formation. Every enabled request is given a ten-bit priority key built from its band's urgency, its intra-band priority and its slot number. The key is the only thing the resolver compares, which is what makes the ordering total and deterministic.

Resolution. A single combinational resolver picks the most urgent eligible source. Eligibility excludes sources already in service, sources the processor has masked, and, when something is already in service, every source that is not strictly more urgent than the interrupt at the top of the nesting stack.

Offer. The winner is registered and presented to the processor as a request line plus a source number. Registering it keeps the source lines off the processor's timing path and gives the pair a stable, consistent value.

Accounting. The claim pushes the source and its key onto the nesting stack and marks it in service; the end of interrupt pops the stack and releases it. The snapshotted key, not the current configuration, is what later preemptions are compared against.

3.2 Block structure

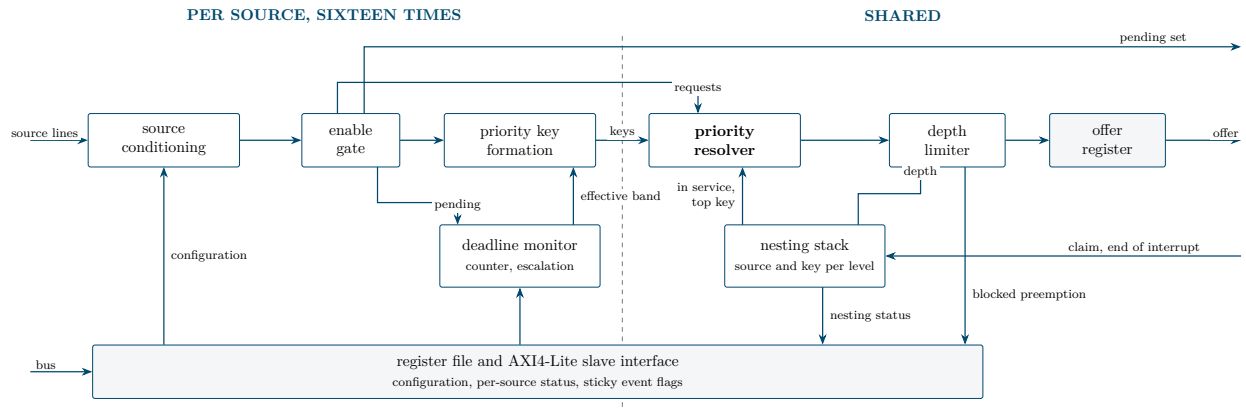


Figure 1: Structure of the controller. Everything left of the dashed line is replicated once per source: conditioning turns a line into a request according to its trigger type and merges the software channel, the enable gate qualifies it, key formation combines the effective band, the intra-band priority and the slot number into one comparable value, and the deadline monitor raises the effective band when a source has waited too long. Right of it, the resolver compares the sixteen keys and picks the most urgent eligible source, the depth limiter withholds that selection when the nesting stack is full, and the offer register presents the surviving result to the processor. Two paths leave the enable gate rather than one: the requests feed the resolver, and the pending set leaves the block directly, because the processor has to see the sources queued behind the winner and not only the winner. The nesting stack is what makes nesting work in both directions — its top entry sets the threshold a source must beat to preempt, and its in-service marking removes a serviced source from its own resolution — and it moves only on the claim and end-of-interrupt pulses. The register file holds the configuration read by the per-source logic, the status written back by it, and the sticky flags that record escalations, spurious claims and preemptions blocked by the depth limit.

3.3 Source life cycle

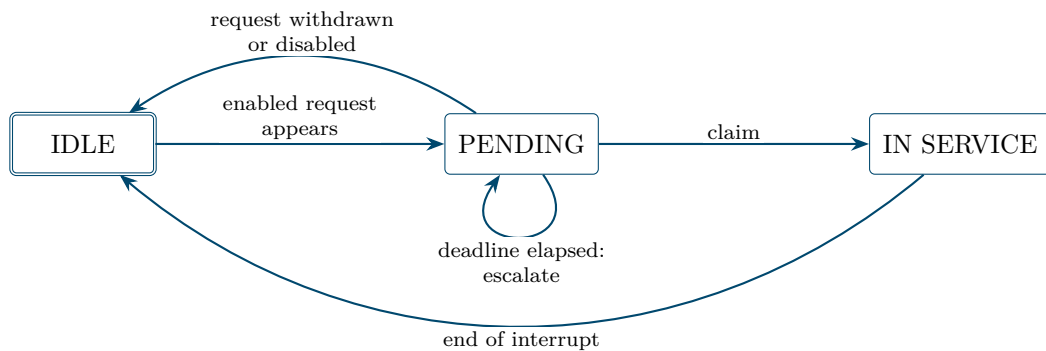


Figure 2: Life cycle of one source. A source becomes pending as soon as it has an enabled request and is not in service; while pending it is visible to the resolver, it appears in the pending set reported to the processor, and its deadline counter runs. Each time the deadline elapses the source escalates and stays pending, which is the only transition that changes its priority. The claim moves it into service, where it occupies one nesting level, stops being eligible for selection and stops counting. The end of interrupt that pops its level returns it to idle. A level source whose line is still high when its handler runs is in service and pending at the same time; it is not offered again, because in-service sources are excluded from resolution, and it returns to being offerable only after the end of interrupt.

3.4 Priority

Priority is a single ten-bit number, compared as an unsigned value.

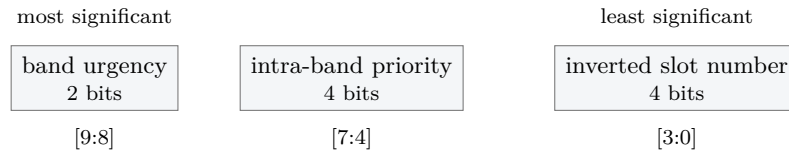


Figure 3: Composition of the priority key. The band’s urgency dominates, so moving a source between bands changes its priority regardless of its intra-band setting. Within a band the four-bit intra-band priority decides, higher being more urgent. The last field is the slot number inverted, which makes the comparison total: two sources with the same band and the same intra-band priority can never tie, and the lower slot number wins. Because no two keys are ever equal, the resolver has exactly one winner in every cycle in which any source is eligible, and a fully specified configuration cannot deadlock on a tie.

A band number is not compared directly. Each of the four bands is given a two-bit urgency in `BAND_CONFIG`, and it is that urgency, not the band number, that enters the key. Software can therefore reorder the whole priority structure with one register write, without touching the per-source configuration. Two bands may be given the same urgency, in which case they merge into one effective group ordered by intra-band priority and slot number.

3.5 Data and control flow

Three flows cross the block and they meet only where the design intends them to.

Configuration enters through the register port and reaches the per-source logic directly. A write lands at the end of the cycle in which it commits, so a source enabled by a write to `INT_ENABLE` is eligible at the very next resolution, and a change of band applies from that resolution onwards. Configuration never passes through the nesting stack, which is why it cannot disturb an interrupt already in service.

Requests flow from the source lines through conditioning and enabling into the resolver, and out through the offer register to the processor. The full set of pending sources leaves the block separately from the offer, because the processor reports what is waiting, not what was selected: the sources queued behind the winner have to remain visible to software.

Accounting flows the other way. The claim and end-of-interrupt pulses move the nesting stack, and the stack feeds back into resolution twice: its in-service marking removes each serviced source from its own eligibility, and its top key sets the threshold every candidate must beat.

4 Interfaces

4.1 Hardware interface

The controller presents four interfaces: clock and reset, the interrupt source inputs, the processor-facing interrupt interface, and one AXI4-Lite slave port. All of them are synchronous to the same clock and all outputs are registered except the AXI4-Lite ready and response-code signals, which the slave front end drives combinatorially from its own state.

Table 3: Port summary

Interface	Signals	Width	Dir.	Purpose
Clock and reset	Clock	1	in	Single clock; asynchronous active-low reset
	Reset	1	in	
Interrupt sources	Source lines	16	in	One line per slot, level or edge according to its configuration
Processor	Enable mask	16	in	The processor's own per-source mask, applied inside the resolution
	Request	1	out	The offer: one prioritised request and the slot it names
	Source number	4	out	
	Pending set	16	out	Every pending source, including those the enable mask holds back
	Claim	1	in	Single-cycle pulses that push and pop the nesting stack
	End of interrupt	1	in	
Register port	Write address	—	—	A complete AXI4-Lite slave; see Table 4
	Write data			
	Write response			
	Read address			
	Read data			

4.1.1 Clock and reset

One clock edge drives the whole block. Reset is asynchronous and active low, and every flip-flop in the block is reset, including the nesting-stack arrays that a valid bit would otherwise cover. The rule applied is that no register may leave reset holding an undefined value, so that a status read taken in the first cycle after release returns a defined value and a bus response code can never be undefined.

Reset is not synchronised inside the block. If the system releases it asynchronously with respect to the clock, the integrator is responsible for the release recovery.

The block leaves reset disarmed: `INT_ENABLE` is zero, so a source line held high across reset produces no request until software enables it.

4.1.2 Interrupt source interface

Sixteen inputs, one per slot, sampled on every clock edge. Their interpretation is per source and is set by `SRCx_CONFIG.TRIG`.

A *level* source requests for as long as its line is high. Nothing is latched, so withdrawing the line withdraws the request, and a source that disappears before it is claimed simply stops being offered. This is the mode intended for a peripheral whose own status register holds the condition until the handler clears it.

An *edge* source is captured on the rising edge of its line into a latch that holds the request until it is claimed. The line may be a single-cycle pulse. Two consequences follow from the latch and both matter in practice. A rising edge that lands in the same cycle as the claim is kept, not swallowed: the claim consumes the request that is on its way to the handler, and the new edge stays latched to be serviced afterwards. And an edge captured while the source is disabled is remembered, so enabling the source presents it immediately; there is no register that discards a latched edge.

The edge latch is qualified by the trigger type but not by the enable. A slot left at its reset configuration is level-triggered and captures nothing, so an unused slot cannot accumulate a stale request; a slot already configured edge-triggered captures every rising edge whether it is enabled or not.

Unused lines must be tied low all the same, because a level-triggered slot whose line is high requests the moment software enables it.

4.1.3 Processor interface

Five signals, and the ordering between them is the interface.

The **enable mask** is an input, not a filter on the output. The processor supplies the mask it will actually honour, and the controller excludes masked sources from the resolution itself. Applying it only to the winner would leave the resolver parked on a source the processor never claims, starving every less urgent source behind it indefinitely. A masked source stays pending: it appears in the pending set, its deadline keeps running, and it is offered on the first resolution after the mask lifts.

The **request** and **source number** are registered together from the same resolution, so the pair is always consistent. The request line is a level, not a pulse: it stays high while an eligible source is waiting and drops when the source is claimed or stops being eligible. The source number is only meaningful while the request is high; when no source is being offered it holds the last number presented.

The **pending set** is combinational and reports every source with an enabled request that is not in service, whether or not the processor's mask would allow it through. It is deliberately not the same information as the offer.

The **claim** pushes the offered source onto the nesting stack. The controller uses the source number as it stood one cycle before the claim, which is what makes the claim refer to the source the processor actually saw even if a more urgent one arrived in the intervening cycle. The **end of interrupt** pops the top entry. An end of interrupt received while the stack is empty is ignored and cannot make the depth underflow.

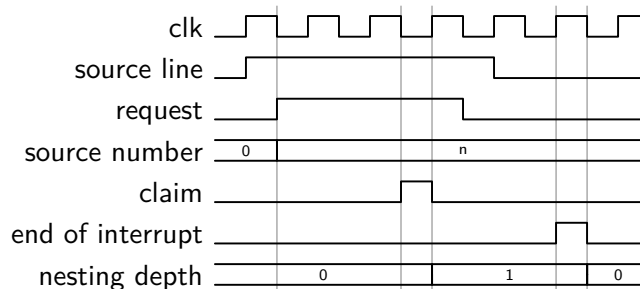


Figure 4: One interrupt from end to end. The source line rises and the request appears one cycle later, together with the source number, because the offer is registered. The processor takes the offer at an instruction boundary of its own choosing and answers with a single-cycle claim in the cycle after the one in which it sampled it. The claim pushes the source, the depth becomes one on the following edge, and the request falls one edge later still, because an in-service source stops being eligible and the offer is registered as well. The handler clears the peripheral, which withdraws the line, and the end-of-interrupt pulse pops the level. The behaviour to observe is that the request goes away on the claim and not on the line falling: it is the in-service marking, not the source, that removes it from the offer.

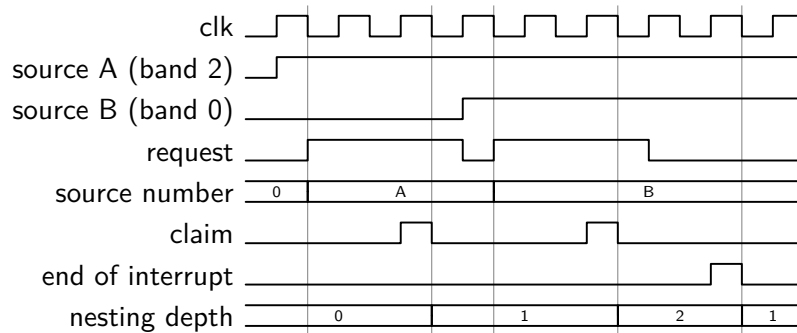


Figure 5: Preemption. Source A is claimed and enters service at depth one, and the request falls because A is no longer eligible against itself. Source B belongs to a more urgent band, so when its line rises its key beats the key snapshot for A and a new offer appears while A’s handler is still running. The second claim pushes rather than replaces and the depth becomes two. The threshold B had to beat is the key recorded at A’s claim and not A’s current configuration, so re-banding A while it is in service cannot change whether B is allowed to preempt it. The end-of-interrupt pulse releases exactly one level: the depth returns to one and A’s threshold is restored, with no separate restore step. A stays in service throughout, and its line being high does not make it offerable again.

4.1.4 Register access interface

An AXI4-Lite slave. The write address and write data channels are collected independently and may arrive in either order; the write response is issued once both have been accepted.

Table 4: Slave port characteristics

Property	Value
Channels	All five
Address and data width	32 bits
Outstanding transactions	One per direction. The address channel deasserts its ready signal until the corresponding response has been taken.
Address decoding	Bits [7:2] only. Bits [1:0] are ignored, so an unaligned address is treated as the word that contains it. Bits above [7] are not examined.
Byte strobes	Honoured. Their effect is per register and is given in Section 4.2.2.
Protection bits	Accepted and ignored. Access cannot be restricted by privilege or security level.
Read latency	One cycle: read data is registered on the address handshake.
Response codes	OKAY for a mapped, permitted access. SLVERR for a read of an unmapped offset, for a write to an unmapped offset, and for a write to a read-only register. A rejected write changes no state, and a rejected read returns zero.
Error reporting	Only through the response code. No error is recorded in a status register.

Rejection is decided from the address alone. A single-byte write to a read-only register is refused exactly as a full-word write to it would be, and a write with no byte lanes enabled at all is still refused if the address is not writable.

4.2 Software interface

4.2.1 Register map

All offsets are relative to the block’s base address. The block occupies 256 bytes; offsets 0xE0 to 0xFC are unmapped and answer **SLVERR**.

Table 5: Register map

Offset	Name	Access	Reset	Function
0x00--0x3C	SRC0..15_CONFIG	RW	0x0000_0000	Per-source configuration
0x40--0x7C	SRC0..15_SW_TRIG	RW	0x0000_0000	Per-source software channel
0x80--0xBC	SRC0..15_STATUS	RO	0x0000_0000	Per-source state
0xC0	BAND_CONFIG	RW	0x0000_001B	Urgency of each band
0xC4	NEST_STATUS	RO	0x0000_0000	Nesting depth and top of stack
0xC8	NEST_MAX	RW, WARL	0x0000_0008	Nesting depth limit
0xCC	ACTIVE_VEC	RO	0x0000_0000	Source currently in service
0xD0	SPURIOUS_LOG	RW1C	0x0000_0000	Sticky spurious claim per source
0xD4	ESCALATION_CFG	RW	0x0000_0000	Escalation mode and target
0xD8	INT_ENABLE	RW	0x0000_0000	Per-source master enable
0xDC	INT_STATUS	RW1C	0x0000_0000	Sticky global event flags

Every reserved bit in the map reads zero. In the two registers that mix reserved bits with writable fields, SRCx_CONFIG and ESCALATION_CFG, a write to a reserved bit is dropped rather than stored, so a reserved bit reads zero even immediately after a write of all ones. A later revision can define one of those bits without breaking software that happened to write a one into it.

4.2.2 Register fields

SRCx_CONFIG, offsets 0x00 to 0x3C, RW. Byte strobes are honoured; unstrobed lanes keep their current contents.

Bits	Field	Access	Description
[31:16]	DEADLINE	RW	Number of cycles the source may remain pending before it escalates. Zero disables escalation for this source and holds its elapsed-time counter at zero. The value is compared against a counter that starts when the source becomes pending, so the first escalation occurs after exactly this many cycles of waiting.
[15:8]	—	—	Reserved. Reads zero; writes dropped.
[7:4]	INTRA	RW	Intra-band priority. A larger value is more urgent. Compared only between sources whose bands have equal urgency.
[3]	—	—	Reserved. Reads zero; writes dropped.
[2:1]	BAND	RW	Band number, 0 to 3. The band's urgency comes from BAND_CONFIG. A change takes effect on the next resolution for a pending source, and does not alter the threshold of a source already in service.
[0]	TRIG	RW	Trigger type. 0 selects level, 1 selects rising edge. Changing this field does not clear a request already latched by the edge detector.

SRCx_SW_TRIG, offsets 0x40 to 0x7C, RW. The software request channel of the slot.

Bits	Field	Access	Description
[31:16]	KEY	write only	Must contain 0xA5A5 and both of its byte lanes must be strobed for a set to take effect. Reads zero.
[15:1]	—	—	Reserved. Reads zero.
[0]	SWREQ	RW	Writing one, with the key present and strobed, raises the software request for this slot. Writing zero clears it; the key is not required to clear. Either write requires byte lane 0 to be strobed. Hardware clears the bit when the slot is claimed, so a software request is consumed like an edge. Reading returns the current state of the channel.

The key exists because a single stray bit written into this register would otherwise inject an interrupt. Requiring the key lanes to be strobed, and not merely to carry the right value, is what makes the guard effective: on a narrow write the lanes that are not strobed carry whatever the master last drove, so checking the value alone could be satisfied by accident.

A software request merges into the same slot as the hardware line. If both are raised, the slot is pending once; the claim clears the software channel and the edge latch, and a level line that is still high keeps the slot pending after the handler returns.

SRCx_STATUS, offsets 0x80 to 0xBC, RO. The only window software has into what a source is doing.

Bits	Field	Access	Description
[31:16]	DDL_TIMER	RO	Cycles elapsed since the source became pending. Runs only while the source is pending and not in service, and only when a non-zero deadline is configured; it is cleared in every other case. After a source has escalated for the last time permitted by <code>ESCALATION_CFG.MULTI</code> the counter stops advancing and holds its value, so it should be read as “time waited, up to the deadline” rather than as a total wait.
[15:6]	—	—	Reserved. Reads zero.
[5:4]	EFF_BAND	RO	The band actually in use. Equal to <code>SRCx_CONFIG.BAND</code> except between an escalation and the end of the wait that caused it.
[3]	SPUR	RO	Set when the claim that put this source into service was spurious. Cleared when that level is released.
[2]	ESC	RO	Set when the source has escalated during the present wait. Cleared as soon as the source stops being pending, which includes the moment it is claimed, so a handler reading this bit for its own source will find it clear.
[1]	ACTIVE	RO	Set while the source occupies a nesting level.
[0]	PEND	RO	Set while the source has an enabled request, independently of <code>ACTIVE</code> . A level source still asserted during its own handler reads <code>PEND</code> and <code>ACTIVE</code> together. Reads zero for a source that is not enabled, even if its line is high, so software cannot mistake a masked peripheral for a quiet one by reading this bit alone.

BAND_CONFIG, offset 0xC0, RW. Four two-bit urgencies, one per band, at bits [1:0], [3:2], [5:4] and [7:6] for bands 0 to 3. Bits [31:8] read zero. Byte strobes are honoured.

The reset value 0x1B gives band 0 urgency 3 down to band 3 urgency 0, which is the natural order. An all-zero value would make all four bands equally urgent and reduce priority to the intra-band field, so this register is one of the two whose reset value is not zero. Reordering takes effect on the next resolution for pending sources and does not change the threshold of anything already in service.

Bits	Field	Access	Description
[16]	OVF	RO	The same storage as INT_STATUS.OVF. It is cleared by writing one to that bit of INT_STATUS; it cannot be cleared here.
[11:8]	TOP_ID	RO	Source at the top of the nesting stack. Reads zero when the depth is zero, which is why ACTIVE_VEC carries a separate valid bit.
[4:0]	DEPTH	RO	Number of interrupts in service, 0 to 16.

NEST_STATUS, offset 0xC4, RO.

NEST_MAX, offset 0xC8, RW, WARL. Bits [4:0] hold the nesting depth limit; the rest read zero. Reset value 8.

A write is taken only if byte lane 0 is strobed, and the value is taken from bits [4:0] of the write data without merging the existing contents. Values outside the range 1 to 16 are clamped: zero becomes one and anything above 16 becomes 16. Zero is excluded because a limit of zero would mask every offer permanently, and 16 is the physical stack depth.

While the depth has reached the limit no source is offered, whatever its priority, and the attempt is recorded in INT_STATUS.OVF. Lowering the limit below the current depth is permitted and is not destructive: nothing is unwound, offers are simply withheld until enough handlers have returned.

ACTIVE_VEC, offset 0xCC, RO. Bit [8] is set while at least one interrupt is in service; bits [3:0] give the source at the top of the stack. A handler can read this register to learn which source it was entered for without consulting the processor's own cause register, and in a nested situation it names the innermost level.

SPURIOUS_LOG, offset 0xD0, RW1C. Bits [15:0], one sticky bit per source, set when a claim of that source was spurious. Bits [31:16] read zero.

Writing a one clears the corresponding bit; writing a zero leaves it alone. Only byte lanes 0 and 1 take part in the clear. A hardware set occurring in the same cycle as a clear of the same bit wins, so an event is never lost to a concurrent acknowledgement.

ESCALATION_CFG, offset 0xD4, RW. One escalation policy applies to every source; the deadline itself is per source. Byte strobes are honoured, and writes to reserved bits are dropped.

Bits	Field	Access	Description
[8]	MULTI	RW	When set, a source may escalate again each time its deadline elapses again. When clear, a source escalates at most once per wait.
[7:5]	—	—	Reserved. Reads zero; writes dropped.
[4]	MODE	RW	0 moves the source to the band named by TARGET. 1 moves it one step up the urgency order.
[3:2]	—	—	Reserved. Reads zero; writes dropped.
[1:0]	TARGET	RW	Destination band when MODE is 0.

INT_ENABLE, offset 0xD8, RW. Bits [15:0], one master enable per source; bits [31:16] read zero. Byte strobes are honoured. This enable is the controller's own and is independent of the mask the processor supplies: the first says the source is allowed to reach the processor at all, the second says the processor is currently willing to take it. A source needs both.

INT_STATUS, offset 0xDC, RW1C. Three sticky event flags. Bits [31:3] read zero. A clear requires byte lane 0 to be strobed, and a hardware set wins over a same-cycle clear.

Bits	Field	Access	Description
[2]	OVF	RW1C	A source was eligible but could not be offered because the nesting depth had reached its limit. Also visible as NEST_STATUS.OVF .
[1]	ESC	RW1C	Some source escalated. Which one is in SRCx_STATUS.ESC , and only while that source is still waiting.
[0]	SPUR	RW1C	Some claim was spurious. Which one is in SPURIOUS_LOG , which is not cleared by clearing this bit.

4.2.3 Programming sequences

Bring-up. Configure before enabling, and enable last.

1. Write **SRCx_CONFIG** for every source in use: trigger type, band, intra-band priority and deadline.
2. Write **BAND_CONFIG** if the default order is not wanted, and **ESCALATION_CFG** if any deadline is non-zero.
3. Write **NEST_MAX** if a limit other than 8 is wanted.
4. Clear **INT_STATUS** and **SPURIOUS_LOG** by writing ones.
5. Write **INT_ENABLE** last.

The order matters for two reasons. A level line that is already high becomes pending as soon as its enable bit is written, so enabling before the priority is configured can produce an offer against a half-written configuration. And the edge detector latches independently of the enable, so any edge that arrived after the slot was configured edge-triggered is delivered as soon as the source is enabled. There is no register that discards a latched edge, so a source whose line may have glitched between its trigger-type write and its enable should have that first, possibly stale, interrupt absorbed deliberately.

Handler. A handler entered for a source should:

1. read `ACTIVE_VEC` to identify the innermost source, if the processor's own cause information is not being used;
2. read `SRCx_STATUS` and return immediately if `SPUR` is set — the peripheral is no longer asserting and there is nothing to service, but the level was accounted and must still be released;
3. clear the condition in the peripheral, which withdraws a level line;
4. return, which produces the end of interrupt that releases the level.

Every claim is accounted, including a spurious one, so every handler entry must be matched by exactly one return. Omitting the return for a claim believed to be spurious leaves a level open, and the threshold it sets keeps every source that is not more urgent from ever being offered again.

Nesting. Nesting is enabled by the processor re-enabling its global interrupt enable inside a handler; the controller needs no configuration for it beyond a `NEST_MAX` above one. Only a source strictly more urgent than the innermost level is offered, so a handler that re-enables interrupts cannot be preempted by an equal or lower priority.

A handler that does this has one obligation of its own, and it belongs to the processor rather than to the controller. The trap return address and the saved status each live in a single register, so a second trap overwrites both. A handler must save them before it re-enables interrupts and restore them immediately before it returns; without that, the outer handler returns to wherever the inner one was interrupted rather than to the code it was itself entered from. The controller's own accounting is unaffected either way — the levels still stack and unwind correctly — which is what makes the omission easy to miss: it corrupts the program's control flow, not the interrupt state.

Software trigger. Write the full word `0xA5A5_0001` to `SRCx_SW_TRIG`. A narrow write that leaves the key lanes unstrobed is accepted on the bus and has no effect. The channel clears itself when the slot is claimed; writing 0 to bit 0 withdraws it before it is claimed.

Reconfiguring a live source. A source's band and intra-band priority may be changed at any time, including while it is in service. The threshold used for preemption is the key recorded when the source was claimed, so a change cannot retroactively make an in-service interrupt harder or easier to preempt; it applies from the source's next wait.

5 Reuse and reusability

No component of the controller is sourced from elsewhere. Its AXI4-Lite slave front end is a separate, reusable element shared with other register-mapped blocks in the system: it owns the two address channels, the response channels and the single-outstanding rule, and exposes a committed write and a combinational read to whatever register file sits behind it. The controller itself is only that register file plus the resolution logic.

The block has no module parameters. Everything that can be configured is configured at run time through the register interface, and the two structural constants are fixed:

Table 6: Fixed structural constants

Constant	Value	Consequence
Source slots	16	Fixes the width of the source, enable and pending ports and the width of the source number.
Nesting stack depth	16	The physical limit <code>NEST_MAX</code> is clamped to. Sixteen levels is also the largest number of distinct sources that can be in service at once, so the stack is never what runs out first.

The controller holds no global state and may be instantiated more than once in a system, but each instance serves one processor: there is one offer, one claim and one end-of-interrupt pulse, and no way to say which processor a source belongs to.

6 Design of submodules

6.1 Source conditioning

6.1.1 Interface description

In: the sixteen source lines, the per-source trigger type, the software channels, the per-source enable, and the claim. Out: one request bit per slot, consumed by the resolver, by the pending set and by the deadline monitor.

6.1.2 Functional description

Each slot forms its request as the trigger-dependent hardware term, merged with its software channel, gated by the enable. The trigger type selects between the line itself and the output of an edge latch.

The edge latch is set on a rising edge of the line and cleared when the slot is claimed. The set has priority over the clear, and that ordering is the design decision in this block. A rising edge arriving in the same cycle as a claim is two distinct events: the request being claimed is already on its way to a handler, and the new one still has to be serviced. Clearing first would discard it with nothing left to show it ever arrived.

The latch is qualified by the trigger type but not by the enable, so an edge is captured whether or not the slot is enabled at the time, and is delivered when software enables it. A slot that has never been configured edge-triggered captures nothing at all. This is the behaviour described under bring-up in Section 4.2.2.

The enable is applied at this point, before anything else looks at the request. A disabled source is invisible everywhere: it is not offered, it does not appear in the pending set, its `PEND` status bit reads zero, and its deadline does not run.

6.2 Priority key formation

6.2.1 Interface description

In: the per-source effective band from the deadline monitor, the per-source intra-band priority, and `BAND_CONFIG`. Out: one ten-bit key per slot.

6.2.2 Functional description

The band is translated into an urgency by indexing `BAND_CONFIG`, and the key is assembled as shown in Figure 3. The translation is what allows the priority structure to be reordered globally:

the resolver never sees a band number, only an urgency, so a source's rank changes the moment `BAND_CONFIG` changes, with no per-source rewrite.

The least significant field is the slot number inverted rather than the slot number itself, so that a lower slot number produces a larger key. This makes lower-numbered sources win ties, and more importantly makes ties impossible: every source has a distinct key in every cycle.

The band that enters the key is the effective band, which is the configured band except while an escalation is in force.

6.3 Deadline monitoring and escalation

6.3.1 Interface description

In: the per-source request and in-service state, the per-source deadline, and `ESCALATION_CFG`.
Out: the effective band, the escalation status bit, the elapsed-time counter and an escalation event pulse.

6.3.2 Functional description

Each slot carries a sixteen-bit counter that runs while the slot is a pending request awaiting service and a non-zero deadline is configured. It is cleared whenever either condition stops holding, so it measures the current wait and not an accumulated one. Being masked by the processor does not stop it: a source held back by the processor's mask is still waiting, and the whole point of the deadline is to notice that it has waited too long.

When the counter reaches the configured deadline the source escalates, provided it has not already escalated during this wait or `MULTI` allows repeats. Escalation moves the effective band and sets the per-source escalation status bit together, so the two views cannot disagree. In jump mode the effective band becomes the configured target. In bump mode it moves one step up the urgency order.

Bump mode is defined against the urgency order and not against the band number, and the distinction is not cosmetic. `BAND_CONFIG` allows the bands to be reordered, and after a reorder the numerically lower band can be the less urgent one; a bump that simply decremented the band number would then demote the source it was supposed to rescue. The implemented rule selects the band with the lowest urgency that is still strictly above the current one — the true next step up — and holds the band when nothing is more urgent. Under the default order this reduces to the next lower band number, so the natural configuration behaves as expected.

Everything resets when the source stops waiting. The effective band returns to the configured band, the escalation bit clears and the counter clears, which happens both when the source is claimed and when its request is withdrawn. A source therefore never carries an escalation from one wait into the next.

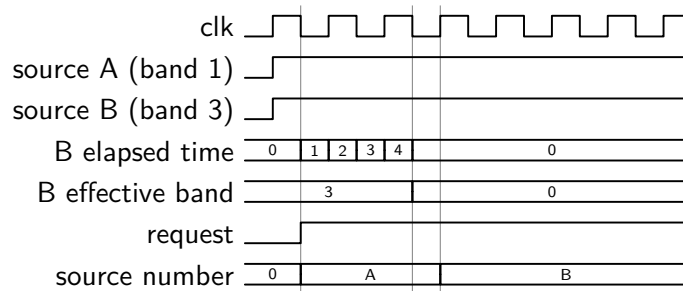


Figure 6: Deadline escalation, with a deadline of five cycles on source B and jump mode targeting band 0. Both sources become pending together; A is in the more urgent band and is the one offered. B's elapsed-time counter advances for each cycle it waits, and on the fifth the escalation fires: B's effective band jumps to the target on the next edge and the counter clears, because the escalation ends the wait it was measuring. B's key now beats A's, so the following edge changes the offer to source B, with neither source line having moved. Escalation only changes which source is offered; it never withdraws a request and never completes one.

6.4 Priority resolver

6.4.1 Interface description

In: the sixteen requests, the sixteen keys, the in-service marking, the processor's enable mask, and the top-of-stack key and depth. Out: the winning source number, its key, and a validity indication.

6.4.2 Functional description

The resolver is combinational and evaluates all sixteen slots in the same cycle. A source is eligible when it is requesting, is not in service, is allowed through by the processor's mask, and — when the stack is not empty — has a key strictly greater than the key at the top of the stack. Among the eligible sources the one with the greatest key wins.

Requiring a strictly greater key is what makes preemption well defined. A source of exactly the same urgency as the interrupt in service is not offered, so a handler cannot be preempted by its own priority level and two sources cannot alternate preempting each other.

Comparing against the top of the stack rather than against a running maximum is what makes nesting unwind correctly: when a level is released the threshold returns to the level below it automatically, with no separate restore step.

The processor's mask is applied here, inside the eligibility test, for the reason given in Section 4.1.3.

6.5 Nesting stack and depth limit

6.5.1 Interface description

In: the claim, the end of interrupt, the winning source number and key, and NEST_MAX. Out: the depth, the top-of-stack source and key, the in-service marking, and the blocked-preemption event.

6.5.2 Functional description

The stack holds one entry per interrupt in service, each entry recording the source number and the key that source had at the moment it was claimed. A claim pushes, an end of interrupt pops, and the depth is the number of occupied entries.

Snapshooting the key rather than recomputing it is the mechanism that keeps nesting consistent under live reconfiguration. If the threshold were derived from the current configuration, moving

an in-service source into a more urgent band would raise the bar for everything queued behind it, and moving it into a less urgent one would allow a preemption that was not permitted when the handler started. With the snapshot, the threshold a handler runs under is fixed at the moment it is entered.

The depth limit is enforced by withholding offers, not by rejecting claims. At the limit the resolver's result is discarded before it reaches the offer register, so the processor is never shown an interrupt it would not be allowed to claim, and the situation is recorded in `INT_STATUS.OVF`. The flag is informational: nothing is lost, the blocked source stays pending with its deadline running and is offered as soon as a handler returns.

An end of interrupt received with an empty stack is ignored, so the depth cannot underflow. A claim received while the depth is already at the limit is ignored entirely — no push, no status change, and no consumption of the edge latch or software channel of the source it names.

6.6 Spurious claim detection

6.6.1 Interface description

In: the claim, the claimed source number, and the per-source request. Out: the per-source spurious status bit, the sticky log bit, and the global spurious event.

6.6.2 Functional description

A claim is spurious when the request of the source being claimed is no longer asserted at the moment the claim lands. This is the case of a level source that withdrew in the window between the offer being sampled and the claim arriving: the processor has already committed to entering the handler and cannot take it back.

A source that is still asserted at the claim and is only cleared later, inside its handler, is legitimate and is not flagged. The test is applied at the claim and nowhere else.

The claim is accounted normally even when it is spurious. The level is pushed, the depth increases, and the handler is expected to return and release it. The alternative — refusing the push — would leave the processor inside a handler with no matching level to release, and the claim and end-of-interrupt counts would diverge permanently. Flagging and accounting keeps the two in step and lets the handler recognise the situation and return early.

Three indications are produced together: a per-source status bit that lives as long as the level does, a sticky per-source log bit that survives until software clears it, and a global sticky event flag.

6.7 Event and status flags

6.7.1 Interface description

In: the escalation pulses, the spurious claim indication and the blocked-preemption indication. Out: the two sticky registers.

6.7.2 Functional description

Both sticky registers apply the same rule: a hardware set in the same cycle as a software clear of the same bit leaves the bit set. An event that occurs while software is acknowledging the previous one is therefore never lost, at the cost of software occasionally seeing a flag it believed it had just cleared.

The global overflow flag is exposed at two addresses over one piece of state, which means it can only be cleared through `INT_STATUS`. Reading it from `NEST_STATUS` alongside the depth and the top of stack is a convenience for a handler that is already reading the nesting state.

The per-source spurious log is not cleared by clearing the global spurious event: the global flag says that something happened, the log says what, and they are cleared independently.

6.8 Register interface

6.8.1 Interface description

In: the AXI4-Lite slave port and, from the register file, a combinational indication of whether an offset is readable and whether it is writable. Out: a committed write with its address, data and byte strobes, and the read address.

6.8.2 Functional description

The write address and write data channels are tracked independently, each with its own accepted flag, and the write commits when both have been captured. This allows a master to present them in either order or in the same cycle. A single commit pulse then carries the address, data and strobes to the register file, which applies them where the address matches.

Both address channels withhold their ready signal while a response is outstanding, which enforces the one-transaction-per-direction limit at the interface rather than relying on the master to respect it.

Whether an access is permitted is decided from the address alone and is independent of the byte strobes. The write commit pulse is produced for a rejected write as well, but no register decodes an address outside its own range, so a rejected write cannot change state.

Reads are registered on the address handshake from a combinational read multiplexer, giving a one-cycle read latency and a stable value on the data channel.

Both response-code flags are reset, which matters on the bus and not only in the reset tree: the response code is driven whether or not the corresponding valid signal is high, so an unreset flag would present an undefined response code from time zero until the first transaction completed.

6.9 RTL organisation

The RTL is grouped by function: interfaces and constants, combinational decisions, state updates, and outputs. Each **always** process assigns one state signal or one array; loop indices are local iteration variables. Priority conditions are explicit, and registers have one procedural owner. Existing parameters retain their function. Architectural widths fixed by RV32I or the sixteen-source PIC contract remain fixed rather than exposing unsupported configurations. RTL files contain no ``timescale`; time units belong to the testbenches or simulator command line.

7 List of bugs and workarounds

No functional defect is currently open against the controller. The entries below are implementation limitations and deliberate restrictions that a system integrator or a driver author needs to know about.

Table 7: Known limitations

Item	Description and consequence
No synthesis result	The block is written within the synthesisable subset, but no synthesis or timing run has been performed. The resolver compares sixteen keys combinationally in one cycle and is the expected frequency limiter; it has not been measured.
No module parameters	The source count and the physical stack depth are fixed at sixteen. A system needing a different source count cannot obtain it by parameterisation.
Address decoding limited to eight bits	Only address bits [7:2] are decoded. A decode window wider than 256 bytes exposes aliases of the entire register file instead of returning an error, so the window must be sized to the block.
Protection bits ignored	The protection field of both address channels is accepted and discarded. Register access cannot be restricted to a privilege or security level.
Claim and end of interrupt not interlocked	Asserting both in the same cycle corrupts the nesting accounting irrecoverably, and nothing in the block prevents it. This is a constraint on the processor, stated in Section 2.
Claim timing is fixed at one cycle	The claim must follow the cycle in which the offer was sampled. A master that claims later will claim whichever source was being offered one cycle before its pulse.
A latched edge cannot be discarded	The edge detector latches independently of the enable, and only a claim clears it. An edge that arrives between a slot being configured edge-triggered and being enabled is therefore delivered once, and software has to absorb it.
Elapsed-time counter stops at the deadline	Once a source has escalated as often as the configuration permits, its elapsed-time counter holds instead of continuing to count. The field measures the wait up to the deadline, not the total wait, and cannot be used to measure a long starvation.
Software trigger clear is unguarded	Setting a software request requires the key; clearing one does not. A stray write of zero to bit 0 of the register withdraws a pending software request.
Nesting depth limit register ignores merging	A write to the depth limit takes its value from the low five bits of the write data and requires byte lane 0; it does not merge with the stored value. A narrow write to a higher lane has no effect.
One escalation policy for all sources	The mode, the target band and the multiple-escalation setting are global. Only the deadline itself is per source, so two sources cannot escalate by different rules.
Single processor per instance	There is one offer and one claim and end-of-interrupt pair. Distributing sources across several processors is not supported.

8 Annex

8.1 Reset state

Reset is asynchronous and active low, and dominant: it returns the block to the state below from any state, including one with interrupts in service, without waiting for a clock edge.

Table 8: Reset state

Element	State after reset
INT_ENABLE	Zero. No source can reach the processor until software enables it.
SRCx_CONFIG	Zero for all sixteen: level-triggered, band 0, intra-band priority 0, no deadline.
BAND_CONFIG	0x1B. Band 0 most urgent through band 3 least.
NEST_MAX	8.
ESCALATION_CFG	Zero: jump to band 0, single escalation.
SPURIOUS_LOG, INT_STATUS	Zero.
Software channels	All clear.
Edge latches	All clear.
Nesting stack	Depth zero; every entry cleared, so the top-of-stack source number and the in-service indication read defined values immediately.
Deadline counters and effective bands	Cleared and tracking the configured band.
Offer	Request low, source number zero.
Bus outputs	Both valid signals low, both response codes OKAY, read data zero.

Only two registers reset to a non-zero value, and in both cases zero would be a wrong setting rather than a neutral one: an all-zero band configuration would give the four bands equal urgency, and a nesting limit of zero would mask every offer permanently.

8.2 Timing summary

Table 9: Latencies, in cycles

From → to	Cycles	Note
Level line high → request presented	1	The offer is registered
Rising edge on line → request presented	2	One cycle to latch the edge
Software trigger write commits → request presented	2	One cycle to store the write
Enable write commits → request presented	2	For a line already high
Offer sampled → claim expected	1	Fixed by the interface
Claim → depth and status updated	1	
End of interrupt → depth updated	1	
Source becomes pending → escalation in effect	DEADLINE	The effective band is updated on the edge that ends the last of those cycles
Read address accepted → read data valid	1	